

Jenkins Dashboard

Release Manager & Auto Deployment Pipeline

Complete Handover Guide for the OASYS Team

Prepared By: Vinayak Bansal · **Team:** OASYS · **Company:** OFBusiness

This document is a complete walkthrough of the Jenkins Deployment Dashboard — its architecture, day-to-day operations, and troubleshooting playbook.

Table of Contents

1. System Overview & Purpose
2. Architecture at a Glance
3. Repository Layout & Tech Stack
4. Where It Runs (Hosting & Access)
5. Deployment & Startup (PM2)
6. Environment Variables (must-know)
7. The Jenkins Dashboard (UI)
8. The Release Manager (manual deploys)
9. The Auto Deployment Pipeline (Scheduler)
10. The Jenkinsfile (`QA-Release-Deployment`)
11. The post-clone Script (`postClone.sh`)
12. Sanity Suites (Jenkins + Automation-Agent)
13. Google Chat Notifications
14. API Endpoints Reference
15. Configuration Files Cheat-Sheet
16. Common Debugging Playbook
17. What to Change When ... (Runbook)
18. Handover Checklist & Contacts

1. System Overview & Purpose

The **Jenkins Dashboard** is a lightweight Node.js/Express application built for the OASYS team to make Jenkins deployments observable, automatable, and safe to trigger. It provides three tightly-coupled capabilities that together replace what used to be a manual, error-prone Jenkins-UI-based process.

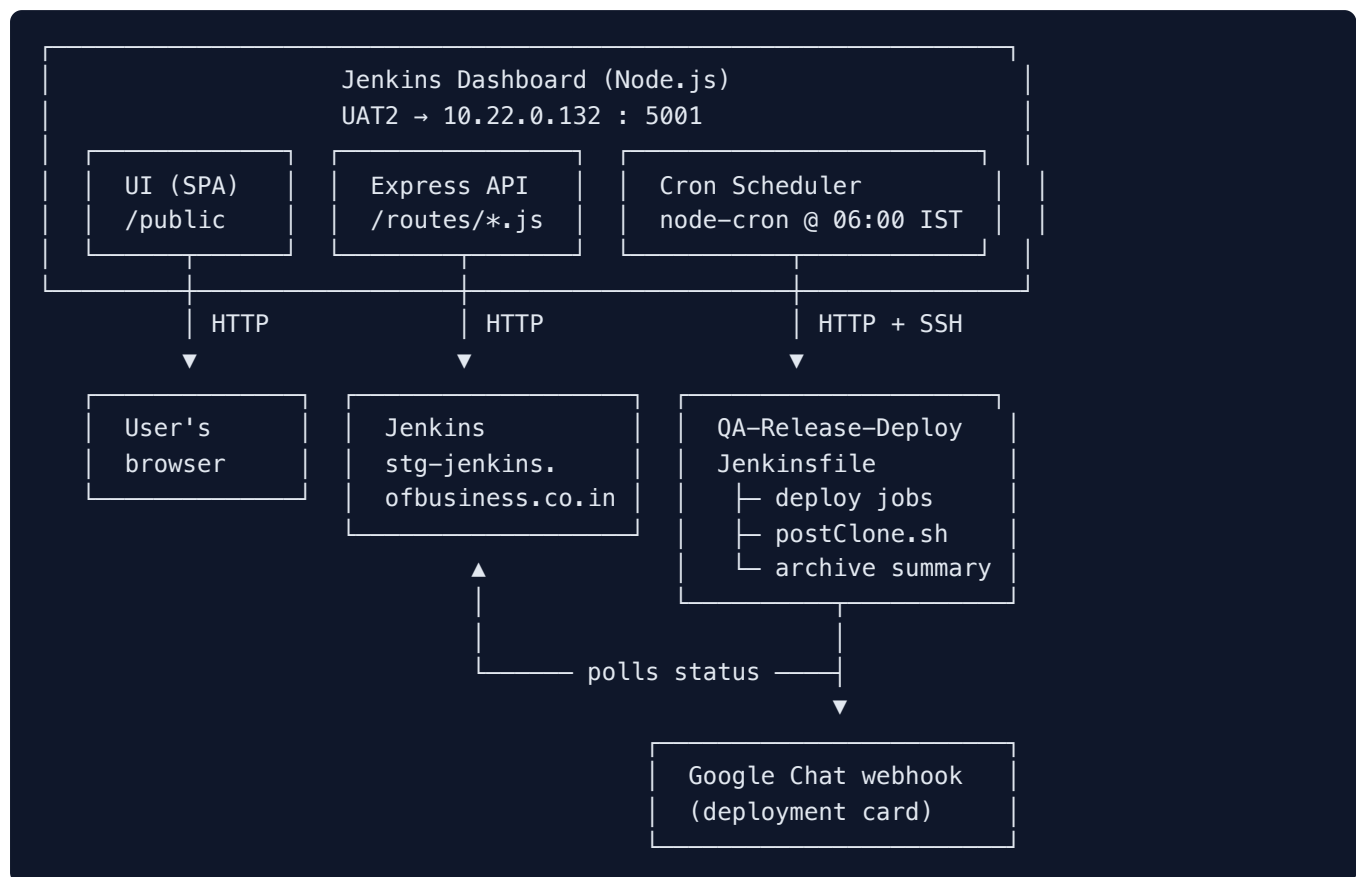
1.1 The three subsystems

Subsystem	Purpose
Jenkins Dashboard (UI)	Live view of every Jenkins deploy job across every staging/UAT environment — branch deployed, last build, running state, server up/down.
Release Manager	UI for triggering an on-demand multi-job release. Under the hood it fires the <code>QA-Release-Deployment</code> Jenkins pipeline.
Auto Deployment Pipeline (Scheduler)	Runs a cron job every morning at 06:00 IST (Mon-Fri & select Saturdays). Triggers the same <code>QA-Release-Deployment</code> pipeline for <code>uat1 / master</code> , waits for it to finish, runs the post-clone hooks, then fires two sanity suites and pushes a Google Chat report.

1.2 What problem does it solve?

- QA no longer has to open Jenkins and click 15+ jobs one at a time.
- UAT1 gets a fresh build of `master` every weekday morning without human involvement.
- Post-deploy fix-ups (Redis, Mongo, whitelisting, role updates) are codified — nobody has to remember them.
- Failures are visible on Google Chat within seconds, not the next day.

2. Architecture at a Glance



Key insight: The dashboard is a *thin client* over Jenkins. It has no database. Everything it shows is fetched from Jenkins' REST + wfapi endpoints in real time. All persistent state lives in Jenkins itself; the dashboard just orchestrates and visualises.

3. Repository Layout & Tech Stack

3.1 Tech stack

Runtime	Node.js (any LTS $\geq$ 16)
Web framework	Express 4
Cron	node-cron (Asia/Kolkata timezone)
HTTP client	node-fetch v2
Config	js-yaml (jobs.yaml), dotenv (.env)
Process manager	PM2 (production)
Frontend	Vanilla HTML/CSS/JS single-page app (no framework)

3.2 Directory tree

```
jobs-dashboard/
├── index.js                ← Express app entry point (starts server + cron)
├── Jenkinsfile            ← The QA-Release-Deployment pipeline definition
├── postClone.sh           ← Post-deploy fix-up script (runs on uat1)
├── deploy.sh              ← Server-side deployment script (git pull + pm2 restart)
├── ecosystem.config.js    ← PM2 process config
├── package.json           ← Dependencies + scripts
├──
├── jobs.config.js         ← Jenkins URL, credentials, JOBS list, env IPs
├── jobs.yaml              ← MASTER manifest of deployable jobs (used by dashboard +
Jenkinsfile)
├── scheduler.config.js    ← Cron schedule, sanity config, retry knobs
├──
├── routes/
│   ├── deploymentRoutes.js ← /api/deployments, /api/trigger-release, /api/build/...
│   └── schedulerRoutes.js  ← /api/scheduler/status, /trigger, /history, /renotify
├──
├── services/
│   ├── jenkinsService.js   ← All Jenkins REST calls (fetch, trigger, abort, logs,
artifact)
│   ├── schedulerService.js ← Cron logic, poll pipeline, sanity queueing, retry
│   ├── chatNotificationService.js ← Google Chat card builder + POST-with-retry
│   └── serverStatusService.js ← Live ping of downstream services (/status endpoints)
├──
├── public/                ← Static SPA served at "/"
│   ├── index.html
│   ├── css/dashboard.css
│   └── js/app.js           ← ~1000 lines: renders dashboard, polls APIs
├──
└── DEPLOYMENT_GUIDE.md    ← Original deploy notes
```

Golden rule: `jobs.yaml` is the *single source of truth* for what gets deployed. Both the Node scheduler AND the Jenkinsfile read it. Never hand-edit the pipeline to add a job — add it here instead.

4. Where It Runs (Hosting & Access)

Host	UAT2 — <code>10.22.0.132</code>
User	root
Install path	/root/jenkins-dashboard
Port	5001 (bound to 0.0.0.0)
Access URL	http://10.22.0.132:5001 (needs OFB VPN if remote)
Process manager	PM2 — process name <code>jenkins-dashboard</code>
Git remote	git@github-office:vinayakbansal-ofb/jenkins-deployment-dashboard.git

Automation-agent

Separate repo, runs under its OWN PM2 process on UAT2.
Path is set via `AUTOMATION_AGENT_PATH` env var.

4.1 SSH in

```
# From within OFB VPN
ssh root@10.22.0.132
cd /root/jenkins-dashboard
```

5. Deployment & Startup (PM2)

5.1 Fresh install on a new server

```
# 1. clone
ssh root@10.22.0.132
git clone git@github-office:vinayakbansal-ofb/jenkins-deployment-dashboard.git \
    /root/jenkins-dashboard
cd /root/jenkins-dashboard

# 2. install
npm install

# 3. install pm2 globally (if missing)
npm install -g pm2

# 4. start with env vars
JENKINS_USER=<user> JENKINS_TOKEN=<token> \
GCHAT_WEBHOOK_URL='https://chat.googleapis.com/...' \
pm2 start index.js --name jenkins-dashboard

# 5. save so PM2 restarts it on server reboot
pm2 save
pm2 startup    # follow the printed instructions
```

5.2 Redeploy after a code change

```
ssh root@10.22.0.132
cd /root/jenkins-dashboard
git pull
npm install                # only if package.json changed
pm2 restart jenkins-dashboard

# Or use the shipped helper:
JENKINS_USER=<user> JENKINS_TOKEN=<token> ./deploy.sh
```

Restart discipline: Restarts are *manual only*. Do NOT wire up file-watch auto-restart. In-memory timers (sanity queue, retry timer) are acceptable losses if the process restarts — the next 06:00 cron run recovers everything.

5.3 Everyday PM2 commands

Status	pm2 status
Live logs	pm2 logs jenkins-dashboard
Last 200 lines	pm2 logs jenkins-dashboard --lines 200 --nostream
Restart	pm2 restart jenkins-dashboard
Stop	pm2 stop jenkins-dashboard
Delete + re-add (env change)	pm2 delete jenkins-dashboard && pm2 start ...

6. Environment Variables (must-know)

Variable	Required?	Purpose
JENKINS_USER	Yes	Jenkins username used for Basic auth on every API call.
JENKINS_TOKEN	Yes	Jenkins API token (Jenkins → User → Configure → Add new token).
GCHAT_WEBHOOK_URL	Recommended	Google Chat incoming webhook. Without this, notifications are logged & skipped.
PORT	No	Default 5001.
SCHEDULER_ENABLED	No	Set to <code>false</code> to disable the daily cron (useful for local dev).
SANITY_ENABLED	No	Set to <code>false</code> to skip the post-deploy Sanity-Suite-OFB run.
AGENT_SANITY_ENABLED	No	Set to <code>false</code> to skip the automation-agent sanity run.
AUTOMATION_AGENT_PATH	No	Override path to the automation-agent repo (default: <code>../automation-agent</code>).

If you rotate the Jenkins token: `pm2 delete jenkins-dashboard` then re-start with the new env vars. `pm2 restart` does *not* pick up new env vars.

7. The Jenkins Dashboard (UI)

Served from `/public`. Single-page app in vanilla JS. Reachable at `http://10.22.0.132:5001`.

7.1 What each view shows

- **Main grid** — one row per (job × environment). Columns include job name, env, branch deployed, last build result, running badge, and quick links to Jenkins.
- **Server status pills** — hits the `/status` endpoint on each downstream service (OFB, Scheduler, File Server, Informer, Orion API, Orion FS) and shows green/red.

8.3 Abort a running release

`POST /api/abort-release` — calls Jenkins' `/stop` on the latest `QA-Release-Deployment` build.

8.4 Renotify (if the browser tab timed out)

Occasionally a release runs longer than the dashboard's page timeout. The *Renotify* button hits `POST /api/scheduler/renotify`, which locks onto the current-running build and fires the Google Chat card when it eventually finishes. Fire-and-forget; the caller gets an immediate `200 OK`.

9. The Auto Deployment Pipeline (Scheduler)

This is the star of the show. Every weekday at **06:00 IST**, a cron fires that:

1. Skips the day if today is a Sunday or 2nd/4th Saturday.
2. Reads the deployable jobs from `jobs.yaml` (excluding `Deploy-Libs`).
3. Snapshots the current `QA-Release-Deployment` build number.
4. Triggers `QA-Release-Deployment` with `uat1` + `master`.
5. If the trigger call itself fails (network / auth), retries once after 30 min.
6. Otherwise polls Jenkins wfapi every 15 s waiting for a strictly-newer build to finish (up to 4 h).
7. Re-verifies each downstream job's real result (because pipeline `propagate:false` lies).
8. Pulls the `postClone-summary.json` artifact and merges its per-step statuses.
9. Sends a Google Chat card with a full breakdown (green/red per stage + per post-clone step).
10. Schedules the Sanity-Suite-OFB Jenkins job to run **30 min later** — always, regardless of pipeline outcome.
11. Schedules the automation-agent sanity batch also **30 min later**.

9.1 Cron expression & timezone

Expression	0 6 * * * (06:00 daily)
Timezone	Asia/Kolkata (hard-coded – see <code>schedulerService.js</code>)
Skip days	Sunday + 2nd & 4th Saturday of the month
Poll interval	15 000 ms (<code>POLL_INTERVAL_MS</code>)
Overall poll timeout	4 hours (<code>POLL_TIMEOUT_MS</code>)
Trigger retry	1 retry, 30 min cool-down
Sanity delay	30 min after pipeline finishes

9.2 Why "poll for a strictly-newer build"?

Jenkins doesn't return the build number synchronously when you trigger via `/buildWithParameters`. It just queues. So the scheduler snapshots the current build number *before* triggering, then polls until a higher number appears and its status is no longer `RUNNING`. This prevents mistakenly reporting on yesterday's build.

9.3 Skipped-day logic

```
const dayOfWeek = now.getDay();           // 0 = Sun, 6 = Sat
const dayOfMonth = now.getDate();

if (dayOfWeek === 0)    → skip (Sunday)
if (dayOfWeek === 6) {   // Saturday
    const nth = Math.ceil(dayOfMonth / 7);
    if (nth === 2 || nth === 4) → skip
}
```

9.4 The trigger-failure retry

If the POST to Jenkins fails (network error, auth error, 5xx), the pipeline never started — so it's safe to try again. The scheduler queues one retry 30 min later. *Post-trigger* errors (poll timeout, pipeline actually failed) never retry; they go straight to the notification path so the team knows.

9.5 In-memory state

All state lives in `schedulerState` inside `schedulerService.js`:

```
{
  enabled, lastRun, nextRun, currentRun,
  pendingSanity, pendingRetry,
  history: []      // last 10 runs
}
```

Exposed via `GET /api/scheduler/status`. This is *intentionally* in-memory — a process restart wipes it, which is fine because the next cron tick recovers everything.

9.6 Manually trigger the scheduler (outside the cron window)

```
curl -X POST http://10.22.0.132:5001/api/scheduler/trigger \
  -H 'Content-Type: application/json' \
  -d '{ "branch": "master", "env": "uat1" }'
```

10. The Jenkinsfile (`QA-Release-Deployment`)

Lives at the repo root. This is the Jenkins declarative pipeline that *actually* triggers downstream deploy jobs. Both the scheduler and Release Manager call this pipeline; they never trigger individual deploy jobs directly.

10.1 Parameters (accepted by the pipeline)

Parameter	Meaning
<code>RELEASE_BRANCH</code>	Branch to deploy (dashboard passes <code>master</code> by default).
<code>STG_ENV</code>	Target environment (<code>uat1</code> , <code>stg1</code> – <code>stg10</code> , <code>uat2</code>).
<code>JOBS_TO_RELEASE</code>	Comma- or newline-separated list of job display names (from <code>jobs.yaml</code>).
<code>LIBS_TO_DEPLOY</code>	Optional list of libs for the <code>Deploy-Libs</code> job.
<code>NOTIFY_EMAIL</code>	Recipient of the HTML email report at the end.
<code>GCHAT_WEBHOOK_URL</code>	Chat card sent by the pipeline (separate from the dashboard's own card).
<code>DRY_RUN</code>	If true, logs what it <i>would</i> trigger but doesn't actually.

10.2 Stages

1. **Initialize & Debug** — resolves env vs params (env wins), loads `jobs.yaml` , parses the job list.
2. **Execute Deployments** — two phases:
 - *Phase 1*: Deploy-Libs (one trigger per library).
 - *Phase 2*: Each service, in the order they appear in `jobs.yaml` . Frontend jobs with `fe_type: standard` get a Website+MSite pair. `fe_type: special` (Merge-FE) gets a 4-run sequence defined in the manifest.
3. **Post Execution / Post Clone** — scps `postClone.sh` to `uat1` (`10.22.0.131`), runs it, scps back `/tmp/postClone-summary.json` , archives it as a build artifact.
4. **Final Reporting** — builds an HTML table + Google Chat card and emails/POSTs it.

10.3 Why `propagate: false` ?

Each downstream `build job:` call uses `propagate: false` and `catchError` . This means one failing job does NOT abort the whole release — every job still gets a chance to deploy. The Node scheduler compensates for the lost signal by *re-checking* each downstream job's real status after the pipeline finishes (see `fetchJobEnvMap` in `jenkinsService.js`).

10.4 Adding a new deploy job to the pipeline

Just add it to `jobs.yaml` — nothing to change in the Jenkinsfile. Keys you can set on each job entry:

<code>name / jenkins_job</code>	display name and actual Jenkins job name (often identical)
<code>type</code>	backend frontend
<code>fe_type</code>	none standard (Website+MSite) special (custom runs[])
<code>no_params</code>	true for jobs like Bheem-Redis-Clone that take no params
<code>default_branch</code>	Override master/main when caller passes the default
<code>branch_param</code>	Non-standard branch parameter name (default: branchName)

<code>env_prefix</code>	Custom prefix if the job's Env value is not <code>ofb_<env></code>
<code>extra_params</code>	Free-form map appended to <code>jobParams</code>
<code>has_libs_param / libraries</code>	Special handling for Deploy-Libs

11. The post-clone Script (`postClone.sh`)

A Bash script that runs on `uat1` (`10.22.0.131`) after every release. It replays a fixed list of "housekeeping" tasks that would otherwise have to be done manually every time UAT1 is re-cloned from prod data.

11.1 What it does — step by step

1. **Pre-checks** — Redis, MySQL, Elasticsearch, MongoDB. If any is down, it tries to start it. If it can't, the step fails but the script continues.
2. Sets `DEVELOPER_KEY_VALUE = 1` in Redis.
3. Sets `VERIFY_LINK_MAX_RETRY_COUNT = 3`.
4. Hits `POST /api/v1/internal/testLogin/<token>?key=1`.
5. Whitelists a fixed list of QA emails via the Notification service.
6. Whitelists a fixed list of QA mobile numbers for SMS/WhatsApp/Android.
7. Prints the currently-deployed branch of OFB / Notification / File Server / Scheduler.
8. Redis: resets all `poSupplierCounter` hash values to 1000.
9. Redis: `del BKALERT 7166`.
10. Mongo: resets `informer.consumerAccessKey`.
11. Updates roles on a fixed set of test-user account IDs.
12. Redis: `SUNIONSTORE jvsrCompanyNameSpaces:ofb ← groupCompanyNameSpaces:ofb`.

11.2 The summary file — `/tmp/postClone-summary.json`

Each step is wrapped in `run_step`, which captures name, status, duration, error message, and an optional "note" (used when a service was restarted). At the end, a JSON array is written to `/tmp/postClone-summary.json` which the Jenkinsfile scps back and archives as a build artifact. The Node scheduler then downloads the artifact and includes each step in the Google Chat card.

Fallback: If the summary file is missing (e.g. script crashed before write, or scp back failed), the scheduler falls back to the wfapi stage status for a coarse Post-Clone result — with a helpful hint in the error field pointing at the likely cause (Redis/MySQL/ES/Mongo).

11.3 Modifying the script

1. Edit `postClone.sh` locally.
2. Commit & push. The Jenkinsfile scps the *current* repo copy to uat1 on every release — you don't have to copy it manually.
3. To add a new step: write a `step_xxx()` function, then a `run_step "Name" step_xxx` line at the bottom.

12. Sanity Suites (Jenkins + Automation-Agent)

Both sanity flows are queued **30 min after** the pipeline finishes, regardless of outcome — because "did anything actually break?" matters even when a deploy partially failed.

12.1 Jenkins sanity — `Sanity-Suite-0FB`

Fired via `triggerJob('Sanity-Suite-0FB', SANITY.PARAMS)`. See `scheduler.config.js → SANITY` for the exact parameter set (`tests: DevTest`, `env: ofb_uat1`, etc.). This is a standard Jenkins job — its result shows up in Jenkins and is emailed by that job itself.

12.2 Automation-agent sanity

The `automation-agent` is a **completely separate application** that lives in its own repo and runs under its **own PM2 process** on UAT2 — parallel to `jenkins-dashboard`. It is the long-running Chrome-extension-driven UI automation service; it does its own work 24/7 whether the dashboard exists or not.

When the sanity delay fires, the dashboard does **not** talk to that running PM2 daemon. Instead it spawns a fresh, short-lived child process by `cd`-ing into the automation-agent's code directory and running `node scripts/run-flow-bot.js --sanity`. That child reads `token.json` and `sanityRunItems` from the same folder, executes the sanity batch, then exits. The always-on automation-agent PM2 daemon is untouched.

```
UAT2 server
├─ pm2: jenkins-dashboard ← this repo
│   └─ spawn('node scripts/run-flow-bot.js --sanity')
│       cwd = AUTOMATION_AGENT_PATH (one-off child, ~few minutes)
└─ pm2: automation-agent ← separate app, always on
    (independent of the sanity child – coexists)
```

Stdout / stderr from the child are piped back and prefixed `[AgentSanity]` in `pm2 logs jenkins-dashboard`. If the child hangs, it's killed after 1 h (`AGENT_SANITY.TIMEOUT_MS`).

Setup on the server: The `automation-agent` repo must already be present on UAT2, and its path must be exposed via the `AUTOMATION_AGENT_PATH` env var to the dashboard's PM2 process. Its `token.json` must be present and valid — that's where the sanity auth & the `sanityRunItems` list live.

Debugging on UAT2: `pm2 status` shows *both* entries. Use `pm2 logs automation-agent` for the always-on daemon and `pm2 logs jenkins-dashboard | grep AgentSanity` for the sanity child.

12.3 Disabling sanity

```
# Skip the Jenkins Sanity-Suite-0FB run
SANITY_ENABLED=false

# Skip the automation-agent run
AGENT_SANITY_ENABLED=false
```

13. Google Chat Notifications

Sent by `services/chatNotificationService.js`. Two-level structure:

- **Header** — overall status emoji (✅/❌/⚠️/🚫) + env, branch, and start/finish timestamps.
- **Deploy stages** — one row per downstream job with 🟢/🔴/🟡/⚪ and a "restarted services" warning if applicable.
- **Post-Clone steps** — grouped separately, one row per `run_step` from `postClone.sh`, showing note or error text.

13.1 Retry policy

Max attempts	3 (WEBHOOK_MAX_RETRIES)
Delay between	5 000 ms (WEBHOOK_RETRY_DELAY_MS)
Timeout	10 s per POST

13.2 Rotating the webhook URL

1. Create a new webhook in the Google Chat space.
2. Update the `GCHAT_WEBHOOK_URL` env var on the server.
3. `pm2 delete jenkins-dashboard && pm2 start ...` (or use `./deploy.sh`).

14. API Endpoints Reference

14.1 Dashboard / Jenkins-proxy

Method + Path	Purpose
GET <code>/api/deployments</code>	One row per (job × env) — powers the main grid.
GET <code>/api/history/:job</code>	Last 20 builds for a job.
GET <code>/api/build/:job/:number</code>	Details for a specific build.
GET <code>/api/server-status?env=uat1</code>	Live pings of downstream <code>/status</code> endpoints.
GET <code>/api/jobs</code>	Manifest from <code>jobs.yaml</code> .
POST <code>/api/trigger-release</code>	Fires <code>QA-Release-Deployment</code> + starts the manual watcher.
GET <code>/api/release-logs?number=&start=</code>	Progressive console text — used by live-log tail.
POST <code>/api/abort-release</code>	Aborts the latest <code>QA-Release-Deployment</code> build.
GET <code>/api/release-status</code>	Current build number + stage list (wfapi).
GET <code>/api/config</code>	Jenkins URL + job list + auth-configured flag.

14.2 Scheduler

Method + Path	Purpose
GET <code>/api/scheduler/status</code>	Enabled / lastRun / nextRun / currentRun / history[10].
GET <code>/api/scheduler/jobs</code>	Jobs the scheduler would deploy on the next run.
POST <code>/api/scheduler/trigger</code>	Manually run the scheduled deploy right now.
GET <code>/api/scheduler/history</code>	Last 10 scheduled runs.
POST <code>/api/scheduler/renotify</code>	Attach a background watcher to the current build and Gchat when it ends.

15. Configuration Files Cheat-Sheet

15.1 `jobs.config.js` — dashboard config

- `JENKINS_BASE_URL`, `JENKINS_USER`, `JENKINS_TOKEN`
- `JOBS` — what shows in the dashboard grid.
- `LOG_BRANCH_JOBS` — jobs whose branch must be scraped from console logs (no branch param).
- `ENV_PARAM_NAMES` / `BRANCH_PARAM_NAMES` — trial list to find the right param.
- `ENV_IPS` — env→IP mapping used for IP-based service pings.
- `SERVICES_CONFIG` — list of downstream services + template URLs.

15.2 `scheduler.config.js` — cron config

- `CRON_SCHEDULE` — the cron string (currently `0 6 * * *`).
- `TARGET_ENV` / `DEFAULT_BRANCH` — `uat1` / `master`.
- `EXCLUDED_JOBS` — `Deploy-Libs` (never auto-deployed).
- `POLL_INTERVAL_MS` / `POLL_TIMEOUT_MS` — 15 s / 4 h.
- `SANITY` — Jenkins post-deploy sanity trigger.
- `AGENT_SANITY` — automation-agent sanity trigger.

15.3 `jobs.yaml` — deploy manifest

Master list of everything deployable. Read by both the Node service AND the Jenkinsfile at pipeline start.

15.4 `ecosystem.config.js` — PM2 config

Runs `index.js`, autorestart on crash, memory limit 1 GB, port 5001. Env vars must be set on the shell before `pm2 start`.

16. Common Debugging Playbook

16.1 Dashboard shows nothing / all jobs are blank

1. `pm2 logs jenkins-dashboard --lines 100 --nostream` — look for `401 Unauthorized`.
2. Jenkins token likely expired or revoked. Generate a new one, `pm2 delete + pm2 start` with the new value.
3. Check network from server: `curl -u USER:TOKEN https://stg-jenkins.ofbusiness.co.in/api/json`.

16.2 06:00 cron didn't fire today

1. Was today a Sunday or 2nd/4th Saturday? Check `shouldSkipToday()` logic.
2. Is the process actually running? `pm2 status`.
3. Was `SCHEDULER_ENABLED=false` set by mistake? Check via `GET /api/scheduler/status`.
4. Server timezone off? The cron is hard-coded to `Asia/Kolkata` in `node-cron`, but the host clock still matters for logging. `timedatectl` to verify.

16.3 Pipeline triggered but Gchat card never arrived

1. Check pm2 logs for `[Chat]` lines — 3 failed retries would be logged.
2. Is `GCHAT_WEBHOOK_URL` set? `GET /api/scheduler/status` → `config.webhookConfigured`.
3. Test the webhook manually: `curl -X POST -H 'Content-Type: application/json' -d '{"text":"ping"}' $URL`.
4. Or hit `POST /api/scheduler/renotify` to re-attach a watcher to the current build.

16.4 Post-clone shows a failed pre-check (Redis / Mongo / ES / MySQL down on uat1)

1. SSH into uat1: `ssh root@10.22.0.131`.
2. Check the specific service — commands are literally the ones in `postClone.sh`:
 - Redis: `redis-cli ping`
 - MySQL: `mysqladmin ping`
 - ES: `pgrep -f org.elasticsearch.bootstrap.Elasticsearch`
 - Mongo: `mongosh --eval "db.runCommand({ping:1})"`
3. Restart it using the same command the script uses (see `check_and_start_*` functions).
4. Re-run the pipeline via Release Manager or `POST /api/scheduler/trigger`.

16.5 A specific deploy job keeps failing

1. Open the Jenkins job directly (link is in the Gchat card).
2. Read its console log — the pipeline's `propagate:false` hides the true error from the wrapper.
3. If it's a "wrong branch param name" problem, add `branch_param: xxx` to that job in `jobs.yaml`.
4. If it's an `Env` value mismatch, add `env_prefix:` or a custom `extra_params`.

16.6 "SSH session hangs when I run ./deploy.sh"

Fixed already (see commit `784a1ed`): the post-clone background service now redirects stdout / stderr to `/dev/null`. If it recurs, ensure any long-running background `&` processes are fully daemonised with output redirected — otherwise the SSH channel waits for their pipes to close.

16.7 Scheduler poll timeout ("Pipeline did not complete within 240 minutes")

1. Jenkins is overloaded / a downstream job hung. Check the Jenkins queue.
2. Manually abort the stuck build in Jenkins. The next cron run will not double-trigger — it checks `currentRun`.
3. Consider bumping `POLL_TIMEOUT_MS` in `scheduler.config.js` if this becomes routine (it's already 4 h).

16.8 Two runs kicked off at once

Can't happen from the cron (only one `currentRun` tolerated). Can happen if you hit `POST /api/scheduler/trigger` during a run — the API will return 409. If it somehow does happen, the pipeline itself has `disableConcurrentBuilds()` so Jenkins will queue the second one.

17. What to Change When ... (Runbook)

You want to ...	File(s) to touch
Change the daily cron time	<code>scheduler.config.js</code> → <code>CRON_SCHEDULE</code>
Change target env or branch for auto-deploy	<code>scheduler.config.js</code> → <code>TARGET_ENV</code> / <code>DEFAULT_BRANCH</code>
Add a job to the dashboard grid	<code>jobs.config.js</code> → <code>JOBS</code>
Add a job to the auto-deploy pipeline	<code>jobs.yaml</code> (add a new entry)
Exclude a job from auto-deploy but keep it in the dashboard	<code>scheduler.config.js</code> → <code>EXCLUDED_JOBS</code>
Add a post-clone step	<code>postClone.sh</code> — new <code>step_xxx()</code> + <code>run_step</code> line
Change sanity test params	<code>scheduler.config.js</code> → <code>SANITY.PARAMS</code>
Rotate Jenkins token	Env var + <code>pm2 delete</code> + <code>pm2 start</code>
Rotate Google Chat webhook	Env var + <code>pm2 delete</code> + <code>pm2 start</code>
Add a new UAT/staging env	<code>jobs.config.js</code> → <code>ENV_IPS</code> + Jenkinsfile <code>STG_ENV</code> choice
Add a new downstream /status service	<code>jobs.config.js</code> → <code>SERVICES_CONFIG</code>
Skip an extra holiday	<code>schedulerService.js</code> → <code>shouldSkipToday()</code>

18. Handover Checklist & Contacts

18.1 Access to transfer

- SSH access to `10.22.0.132` (UAT2) — user `root`.
- Jenkins account with permission to trigger `QA-Release-Deployment` and its downstream jobs.
- GitHub access to the `jenkins-deployment-dashboard` repository (currently under `vinayakbansal-ofb`).
- Google Chat space owner rights (to rotate the webhook if needed).
- Access to the `automation-agent` repo (separate app, has its own PM2 process on UAT2) and its `token.json` file.

18.2 First-week checklist for the person taking over

1. SSH in, run `pm2 status` and `pm2 logs jenkins-dashboard` — get comfortable with the log format.
2. Open `http://10.22.0.132:5001` — click through every job in the grid.
3. Trigger a dry-run release from the Release Manager UI (`DRY_RUN=true`).
4. Read `schedulerService.js` top-to-bottom (~670 lines, well-commented).
5. Skim the Jenkinsfile and follow one job through `jobs.yaml` to see how manifest keys become build parameters.
6. Verify a Gchat card lands — `POST /api/scheduler/renotify` against any running build.
7. Watch a real 06:00 IST cron run end-to-end. Trace: cron → trigger → poll → notify → sanity queue.

18.3 Files you WILL touch. Files you probably WON'T.

Touch often	<code>jobs.yaml</code> , <code>scheduler.config.js</code> , <code>postClone.sh</code>
Touch sometimes	<code>jobs.config.js</code> , Jenkinsfile (only for structural changes)
Rarely	<code>services/*.js</code> — battle-tested, avoid unless there's a real bug
Never	Env vars in code — always via shell + PM2

18.4 Contacts

Owner (outgoing)	Vinayak Bansal — <code>vinayak.bansal@ofbusiness.in</code>
Team	OASYS
Company	OFBusiness
Repo	<code>vinayakbansal-ofb/jenkins-deployment-dashboard</code>
Companion repo	<code>automation-agent</code> — separate app, its own PM2 process on UAT2 (used for sanity flows)

Final note. The system is intentionally small and stateless. When in doubt, restart it — the next cron tick recovers everything. Trust the code; it has been running reliably for months. Good luck!